

ARREGLOS Y PUNTEROS

Elaborado por: Juan Miguel Guanira Erazo

PUCP

Contenido del capítulo:

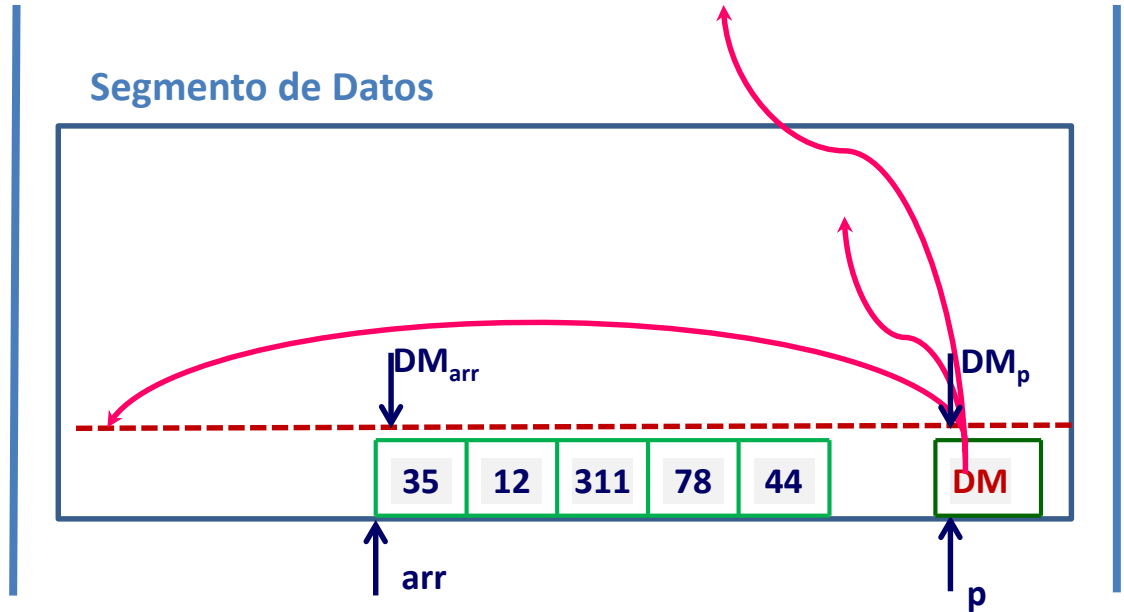
- Conceptos
- Métodos eficientes de asignación dinámica de datos
- Punteros a punteros
- Punteros genéricos
- Punteros a funciones
- Argumentos en la línea de comandos

INICIALIZACIÓN DE UN PUNTERO

```
int arr[5];  
sizeof(arr); → 20
```

```
int *p;  
sizeof(p); → 4
```

Id	DM
arr	DM _{arr}
p	DM _p



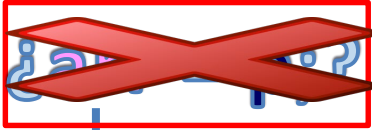
`int *p;`  `p = nullptr;`

Inicialización estática:

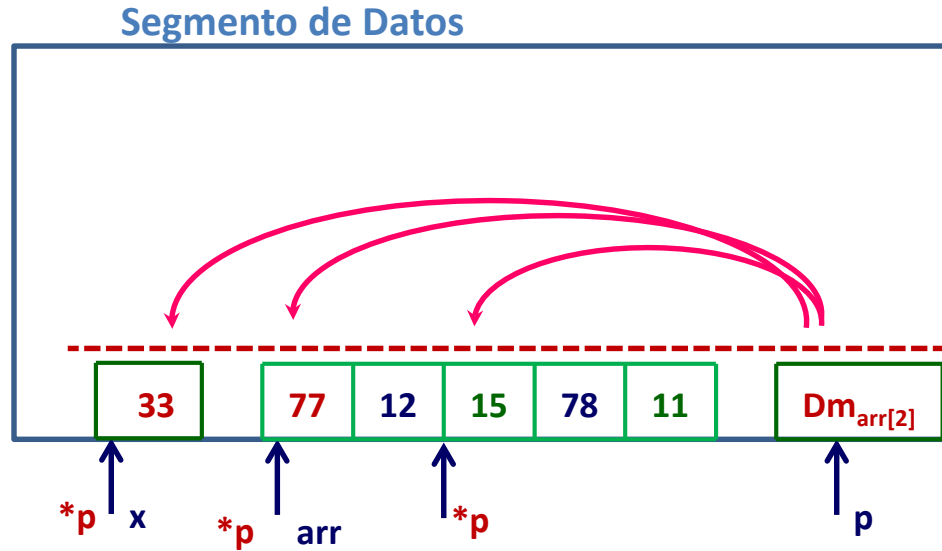
`int x = 27, arr[5];`

`p = &x;`

`p = arr;`



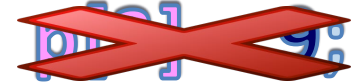
```
p = &x;  
*p = 33;
```



INICIALIZACIÓN ESTÁTICA:

```
p = arr;  
p[0] = 77;  
p[2] = 5;
```

```
p = &arr[2];  
p[0] = 15;  
p[2] = 11;
```



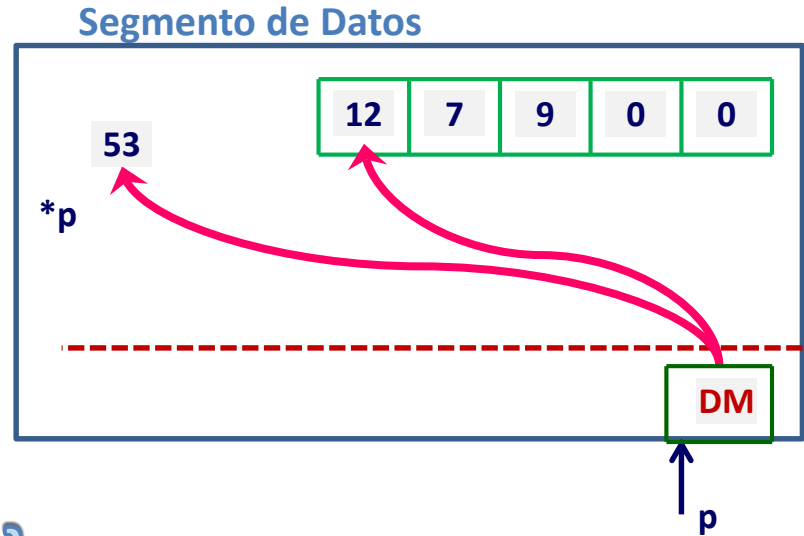
INICIALIZACIÓN DINÁMICA:

`p = new int;`

`p = new int {53};`

`p = new int[5];`

`p = new int[5] {12,7,9};`



ERRORES COMUNES EN EL USO DE PUNTEROS DENTRO DE FUNCIONES:

**GRAVE
ERROR DE
CONCEPTO**

... f (int * &p){ ←

int arr[5];

→ p = arr; ←

...

→ return p; // o return arr;

}

LIBERACIÓN DE UN PUNTERO

```
int *p;
```

```
p = new int;
```

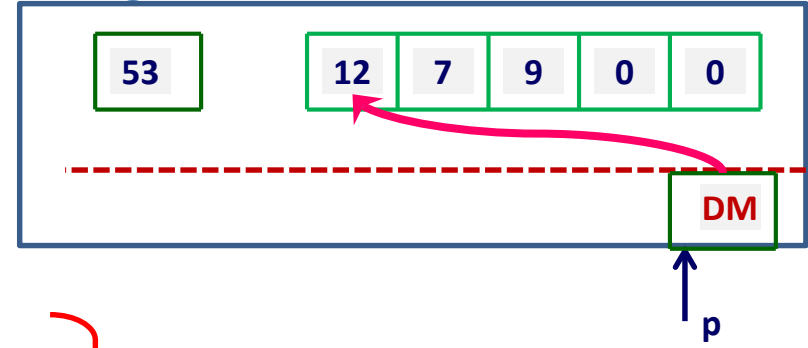
```
p = new int[50];
```



```
*p; o p[i];
```



Segmento de Datos



```
delete p;
```



```
*p; o p[i];
```



EFFECTO DE LA CLAUSULA **CONST** EN LAS REFERENCIAS Y PUNTEROS USADOS COMO PARÁMETROS

```
struct St a;  
f (a);
```

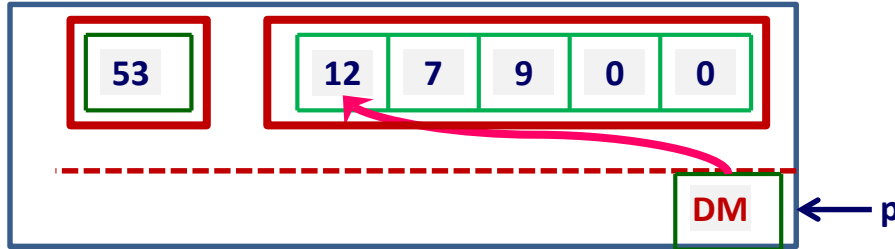
```
... f (struct St &a){  
    a.c = 33;  
    ...  
}
```

```
... f (const struct St &a){  
    cout<<a.c;  
    a.c = 33;  
    ... g (a);  
}
```

```
... g (struct St &a){...
```

```
... f (int *p){  
    *p = 33; ✓  
    P[3] = 21; ✓  
    p = new int; ✓  
    ...
```

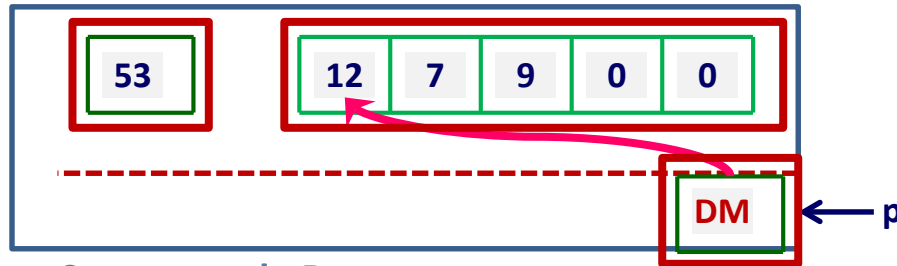
```
... f (const int *p){  
    cout << *p; ✓  
    *p = 33; ✗  
    P[3] = 21; ✗  
    p = new int; ✓  
    ...
```



Segmento de Datos

```
... f (int *const p){  
    cout << *p; ✓  
    *p = 33; ✓  
    P[3] = 21; ✓  
    p = new int; ✗  
    ...
```

```
... f (const int *const p){  
    cout << *p; ✓  
    *p = 33; ✗  
    P[3] = 21; ✗  
    p = new int; ✗  
    ...
```



Segmento de Datos

CADENAS DE CARACTERES

Biblioteca a emplear en el curso:

#include <cstring>

~~#include <cstring.h>~~

~~#include <string>~~

ASIGNACIÓN DINÁMICA DE MEMORIA

El manejo de grandes cantidades de datos en la memoria del computador, hace que sea necesario manejarlos con eficiencia.

Por esto, en esta parte del curso analizaremos dos métodos de almacenamiento, ambos dinámicos, que permitan un mínimo de desperdicio de memoria.

Los métodos que presentaremos los he denominado:

1. Método de asignación exacta de memoria
2. Método de asignación de memoria por incrementos.

MÉTODO DE ASIGNACIÓN EXACTA DE MEMORIA

Cuando se quiere leer un conjunto de datos y almacenarlos en la memoria, por lo general pensamos en un arreglo como:

```
int arr[100], numDat;
```

Sin embargo, si no nos ponemos a pensar que en casi todos los casos que lo usemos, el arreglo no se llenará, por lo que se requerirá una variable auxiliar que nos diga cuantos datos colocamos en el arreglo.

Pero entonces, ¿cómo definir un arreglo cuya capacidad sea la misma que la cantidad de datos que se va a leer?

Bueno, mientras no se invente un lenguaje de programación con instrucciones que lean la mente del usuario, vamos a tener que utilizar un artificio que aproveche el comportamiento de las variables en una función.

Este comportamiento se refiere al hecho que cualquier variable, incluso un arreglo estático, definido dentro de una función “desaparece” cuando la función termine. Esto no sucede con los espacios gestados dinámicamente.

Entonces lo que pretendo hacer es definir, en la función que requiera almacenar los datos, las siguientes variables:

```
int *arr, numDat = 0;
```

Este puntero **arr** almacenará finalmente los datos que se van a leer, pero el proceso de lectura no se realizará en esta función, sino en otra que reciba el puntero por referencia y que cuando retorne haya gestado la memoria justa para los datos que se leyeron también allí.

Esta función se invocará de la siguiente manera:

```
leerDatos (arr, numDat );
```

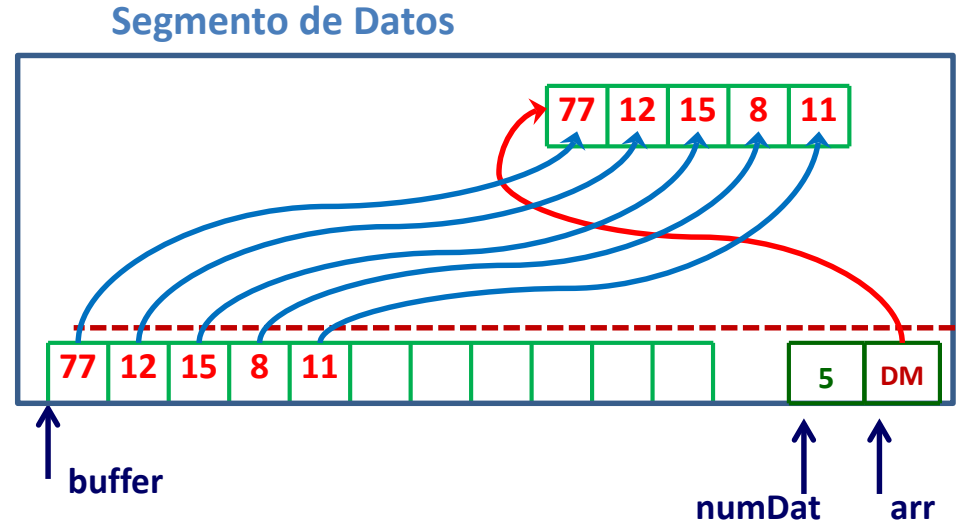
El encabezado de la función será:

```
void leerDatos (int * &, int &);
```

En la implementación de esta función se realizará el siguiente proceso:

```
void leerDatos (int * &arr, int &numDat){
```

1. Se define un arreglo estático lo suficientemente grande (buffer).
2. Se leen todos los datos en el buffer.
3. Se gestiona para **arr**, un espacio de memoria que contenga exactamente a los datos.
4. Se copian allí los datos.
5. Finaliza la función.



EL “MÉTODO EXACTO” es adecuado cuando se pueda determinar de manera aproximada un máximo del número de datos para el arreglo.

MÉTODO DE ASIGNACIÓN DE MEMORIA POR INCREMENTOS

Este segundo método no busca exactitud, lo que busca es tener un mínimo de desperdicio.

Al igual que le método anterior, en la función que requiera almacenar los datos se definen las siguientes variables:

```
int *arr, numDat = 0;
```

Y la invocación y encabezado de la función serán iguales:

```
leerDatos (arr, numDat );
```

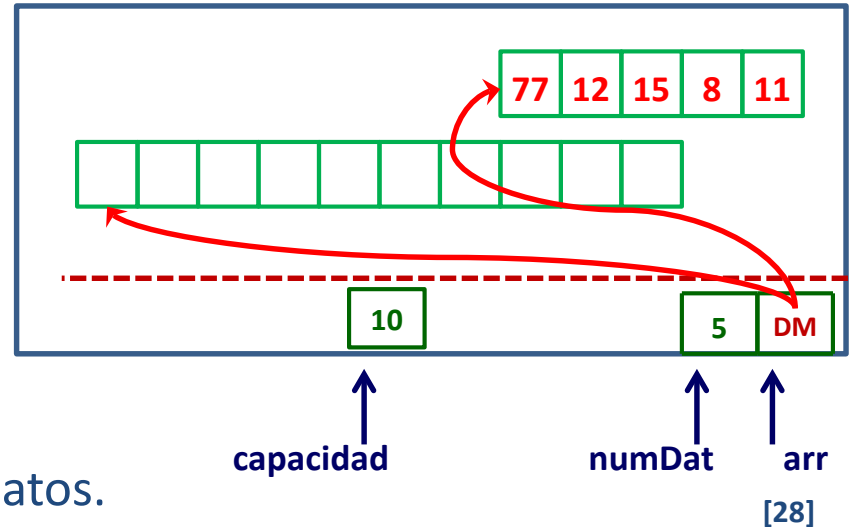
```
void leerDatos (int * &, int &);
```

En la implementación de esta función se desarrollará empleando enteramente memoria dinámica, proceso el siguiente :

```
void leerDatos (int *&arr, int &numDat){
```

1. Se define para **arr** un espacio de memoria reducido.
2. En una variable auxiliar se guarda la capacidad del arreglo.
3. Se leen los datos.
4. Cuando el número de datos se iguale a la capacidad, se gesta un nuevo arreglo de mayor capacidad .
5. Se copian allí los datos.
6. Se libera el arreglo antiguo y se hace apuntar **arr** al nuevo.
7. El proceso se repite hasta leer todos los datos.

Segmento de Datos

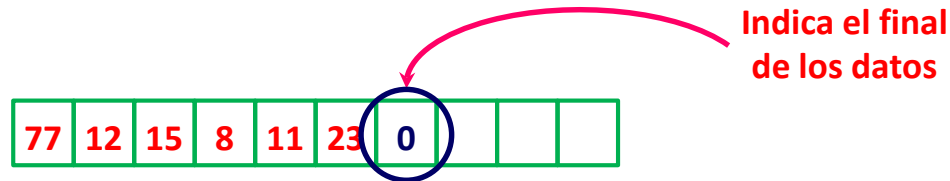


EL “MÉTODO POR INCREMENTOS” es adecuado cuando el número de datos de una ejecución a otra difiere mucho y no se pueda determinar un máximo del número de datos para el arreglo.

VARIANTES A ESTOS MÉTODOS

Se pueden presentar situaciones en las que no se pueda devolver el número de datos que se leyeron.

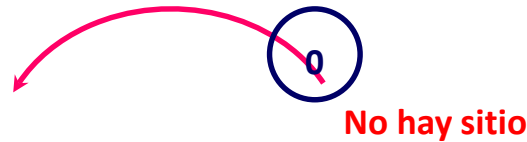
Para esto se opta por colocar marcas al final de los datos, como se hace por ejemplo con el cero (0) en las cadenas de caracteres.



En el caso que se quiera aplicar el método por incrementos y no se pueda retornar el número de datos, colocar la marca final puede ser un dolor de cabeza.

La razón es porque no se puede simplemente colocar el **0** al final de los datos, porque éste podría salirse de la capacidad del arreglo.

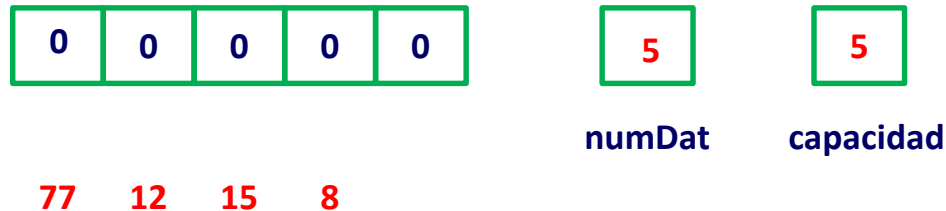
77	12	15	8	11	23	17	31	5	19
----	----	----	---	----	----	----	----	---	----

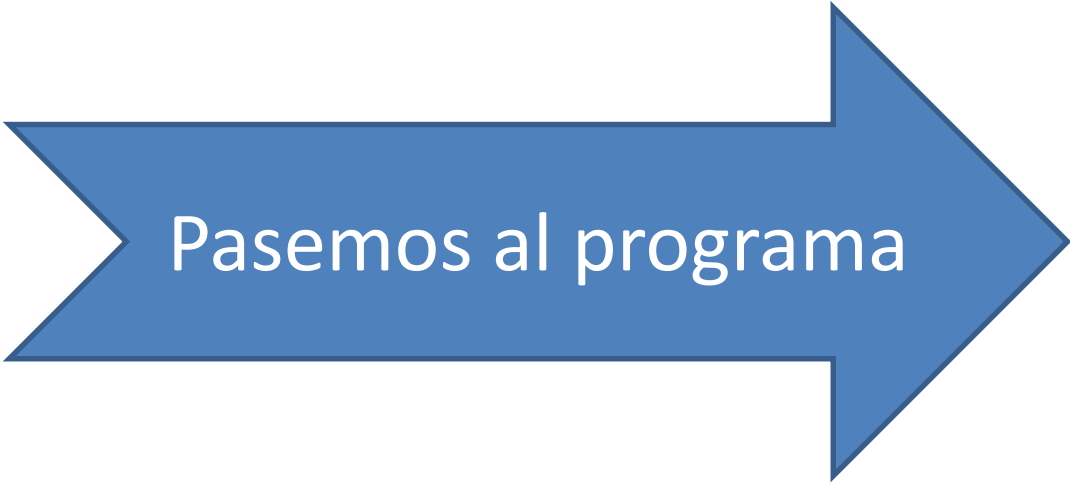


Nos obliga por lo tanto a verificar al final del proceso de lectura y si se ha colmado el arreglo, se debería repetir el proceso de incrementar la capacidad del arreglo para colocarlo.

Por lo que propongo otra forma de manipularlo:

1. Cuando llegue el primer dato, se gesta memoria de acuerdo al incremento y se considera la marca de fin de datos (0) como un dato más que ingresa al arreglo.
2. Se coloca éste en el arreglo seguido del 0.
3. Conforme llegue un nuevo dato, el 0 se desplaza y en su lugar se coloca el nuevo dato.





Pasemos al programa